Politechnika Warszawska
Wydział MiNI

DOKUMENTACJA PROJEKTOWA
SYSTEMU SYMULACJI AUTONOMICZNEGO AGENTA
OPARTEGO NA IMPULSOWYCH SIECIACH NEURONOWYCH (SNN)

Przedmiot: Zaawansowane programowanie obiektowe i funkcyjne (Java)

Rok akademicki: 2025/2026

Autorzy:

1. Maciej Rozpędek
2. Jakub Ramsza

Data: 08.12.2025 – 06.02.2026

Wersja: 0.1

2. Opis tematu

Przedmiot projektu

Głównym celem projektu jest stworzenie zaawansowanej platformy symulacyjnej, w której autonomiczny agent („mysz”) funkcjonuje w wirtualnym środowisku 2D. Unikalnym aspektem systemu jest sterowanie agentem za pomocą Impulsowej Sieci Neuronowej (SNN - Spiking Neural Network) opartej na modelu Izhikevicha [1], naśladującej biologiczne procesy zachodzące w mózgach zwierząt, w przeciwieństwie do klasycznych algorytmów sterowania.

Otwartość i rozszerzalność systemu

Jednym z głównych celów projektu jest stworzenie architektury otwartej na rozbudowę. System nie jest ograniczony do predefiniowanej logiki zachowań, lecz stanowi uniwersalny framework do testowania modeli neurobiologicznych.

Projekt zakłada całkowite odseparowanie „mózgu” silnika SNN od „ciała” (sensorów i efektorów). Dzięki temu system umożliwia:

- **Łatwą integrację nowych zmysłów:** Użytkownik może w prosty sposób zdefiniować nowe mechanizmy analizy otoczenia (np. dodać zmysł słuchu, węchu czy wykrywanie przeszkód) i podłączyć je do sieci neuronowej jako nowe źródła prądu wejściowego.
- **Definiowanie nowych zachowań:** Wyjścia sieci neuronowej mogą być interpretowane w dowolny sposób, co pozwala na łatwe dodawanie nowych akcji (np. wydawanie dźwięków, zmiana koloru, interakcja z innymi obiektami) bez konieczności ingerencji w silnik symulacji.

Cel naukowy i praktyczny

Podstawowym celem projektu jest udostępnienie narzędzia, które znacząco upraszcza proces eksperymentowania z architekturą impulsowych sieci neuronowych. W odróżnieniu od czysto matematycznych bibliotek obliczeniowych, system ten kładzie nacisk na natychmiastową weryfikację hipotez – umożliwia użytkownikowi szybkie testowanie różnorodnych konfiguracji sieci oraz parametrów neuronów bez konieczności ingerencji w kod źródłowy. Wymiar badawczy koncentruje się na możliwości bezpośredniej, wizualnej obserwacji procesu uczenia się w czasie rzeczywistym, co pozwala na intuicyjną ocenę, jak dobór konkretnej topologii wpływa na zdolności adaptacyjne agenta.

Grupa docelowa

System adresowany jest do szerokiego grona odbiorców zainteresowanych tematyką sztucznej inteligencji inspirowanej biologicznie:

- **Studenci i wykładowcy:** Aplikacja stanowi pomoc dydaktyczną w zakresie dokładnych biologicznie impulsowych sieci neuronowych (SNN). Pozwala na wizualizację, w jaki sposób zdefiniowana topologia połączeń i dynamika całej sieci przekładają się na konkretne decyzje, uczenie i ruchy agenta w środowisku.
- **Projektanci modeli neuronowych:** Platforma jest skierowana do osób tworzących nowe lub modyfikujących istniejące modele SNN. Umożliwia weryfikację wpływu struktury połączeń i doboru typów neuronów na proces uczenia się i wydajność sieci.
- **Programiści i hobbyści:** System obniża próg wejścia w tematykę SNN, która często bywa trudna ze względu na barierę matematyczną. Udostępnione narzędzie pozwala na intuicyjne zapoznanie się z praktycznym aspektem tej technologii poprzez edycję parametrów i bezpośrednią obserwację wyników, co stanowi doskonałe uzupełnienie teoretycznej wiedzy zdobywanej z literatury.

3. Założenia projektowe

Wymagania funkcjonalne

1. **Symulacja SNN:** System musi implementować model neuronu Izhikevicha oraz przesyłanie impulsów (spikes) między neuronami.
2. **Konfigurowalność:** Użytkownik może definiować strukturę mózgu (warstwy, typy neuronów, reguły połączeń) za pomocą zewnętrznego pliku YAML.
3. **Środowisko 2D:** Symulacja obejmuje planszę z agentem i losowo pojawiającym się jedzeniem.
4. **Sensoryka i Motoryka:** Agent posiada "wzrok" (sektorowe wykrywanie jedzenia) oraz napęd (sterowanie prędkością i skrętem na podstawie aktywności neuronów).
5. **Uczenie się:** System wspiera mechanizm nagrody (DA-STDP) – po zjedzeniu jedzenia wagi synaptyczne są modyfikowane, co wzmacnia zachowania prowadzące do sukcesu.
6. **Interfejs WWW:** Podgląd symulacji w czasie rzeczywistym w przeglądarce internetowej.

Wymagania niefunkcjonalne

1. **Wydajność:** Silnik SNN musi być w stanie obsłużyć tysiące neuronów w czasie rzeczywistym (ok. 60 FPS).

2. **Modularność:** Łatwe dodawanie nowych typów sensorów (wejść) i efektorów (wyjść).
3. **Technologia:** Backend oparty na Java/Spring Boot, Frontend lekki (bez ciężkich frameworków JS).
4. **Niezależność technologiczna:** Algorytmy neuronowe zaimplementowane od podstaw w języku Java, bez wykorzystania frameworków takich jak TensorFlow czy DL4J.

4. Architektura systemu

System został zaprojektowany w architekturze Klient-Serwer.

1. **Warstwa Danych (Konfiguracja):**
    - Pliki YAML definiują typy neuronów, grupy (warstwy), połączenia, wejścia i wyjścia.
    - Parser („*NetworkTopologyLoader*”) przetwarza te pliki na struktury obiektowe reprezentujące sieć („*SnnNetworkData*”).
2. **Warstwa Logiki Biznesowej (Backend - Spring Boot):**
    - **SimulationEngine:** Główna pętla sterująca czasem symulacji i wątkami.
    - **World:** Model przechowujący kompletny stan środowiska wraz z obiektami fizycznymi oraz zarządzający ich dynamiką i wzajemnymi interakcjami.
    - **Agent:** Reprezentuje autonomicznego agenta, integruje logikę neuronową z modelem fizycznym.
    - **SnnEngine:** Silnik obliczeniowy realizujący symulację matematyczną impulsowej sieci neuronowej.
    - **Input/Output System:** Interfejsy pośredniczące w wymianie sygnałów między środowiskiem a siecią neuronową, zapewniające izolację logiki sterowania od modelu fizycznego.
3. **Warstwa Prezentacji (Frontend):**
    - **SimulationController:** Wystawia punkty końcowe REST API oraz strumień zdarzeń SSE (Server-Sent Events).
    - **Klient WWW:** Odbiera i wizualizuje stan świata w przeglądarce.

Flow w systemie:

Konfiguracja → Start → [Sensory → SNN → Motoryka → Świat → Wizualizacja] → Stop

5. Wykorzystane technologie

Backend

- **Język:** Java 21
- **Framework:** Spring Boot 3.5.9.
- **Biblioteki pomocnicze:**
    - **SnakeYAML:** Parsowanie zaawansowanych plików konfiguracyjnych YAML.
- **Zarządzanie projektem:** Maven, git.

Frontend

- **Język:** Vanilla JavaScript (ES6+)
- **Struktura i styl:** HTML5, CSS3
- **Komunikacja:** Fetch API, Server-Sent Events
- **Rendering:** Canvas 2D API

6. Opis implementacji

Implementacja systemu została zrealizowana z naciskiem na modularność i separację odpowiedzialności, co ułatwia rozszerzanie funkcjonalności.

Komunikacja z frontendem:

„*SimulationController*” udostępnia endpointy „*/api/start*”, „*/api/stop*”, „*/api/speed*” oraz strumień SSE „*/api/stream*” (JSON ze stanem świata). Frontend (Vanilla JS + Canvas 2D) odbiera zdarzenia SSE, renderuje scenę i aktualizuje telemetry.

Wydajność jest osiągnięta poprzez optymalizację operacji na tablicach neuronów i synaps, umożliwiając symulację tysięcy neuronów w czasie rzeczywistym.

Konfiguracja sieci:

Plik „*SNNConfig.yaml*” definiuje strukturę sieci neuronowej. Szczegółowy schemat konfiguracji (sekcje „*neuron_types*”, „*groups*”, „*connections*”, „*inputs*”, „*outputs*”) opisany jest w pliku „*SNN_CONFIGURATION.md*”. Parser („*NetworkTopologyLoader*”) przy użyciu biblioteki SnakeYAML przetwarza plik na obiekty „*SnnNetworkData*”, budując hierarchiczną strukturę grup neuronów oraz reguły połączeń (z obsługą wildcardów [i]).

Silnik SNN („*SNNEngine*”):

Autorski silnik realizuje model Izhikevicha [1] z krokiem czasowym Δt = 1 ms. Równania różniczkowe:

$$v' = 0{,}04v^2 + 5v + 140 - u + I$$
$$u' = a(bv - u)$$

$$if\ v \geq 30mV$$
$$then\ v \rightarrow c, u \rightarrow u + d$$

Neuron generuje spike przy v ≥ 30 mV, po czym następuje reset i przesłanie sygnału do neuronów docelowych. Następnie aktualizowane są wagi zgodnie z zasadą DA-STDP na podstawie obecnego poziomu dopaminy i wartości eligibility traces.

Wejścia i wyjścia:

System wejść/wyjść jest w pełni rozszerzalny dzięki wzorcowi fabryki i interfejsom. Klasa „*Agent*” posiada zmienne „*InputSystem*” i „*OutputSystem*” przechowujące wybrane wejścia/wyjścia.

- **Wejścia (inputs):** Klasy implementujące interfejs „*InputStrategy*”. Metoda „*calculateCurrents*” zwraca tablicę prądów do wstrzyknięcia do wybranych neuronów na podstawie stanu świata i agenta. Aktualnie zaimplementowana jest „*GaussianVisionStrategy*” (sektorowe wykrywanie jedzenia wykorzystujące rozkład normalny).
  Jak dodać własny typ wejścia:
  1. Utwórz klasę implementującą „*InputStrategy*”.
  2. W klasie „*InputStrategyFactory*” dodaj nowy case w metodzie switch. Powinien on mapować unikalny klucz z pliku konfiguracyjnego (np. "MOJE_WEJSCIE") na dedykowaną metodę tworzącą obiekt Twojej klasy.
  3. Stwórz prywatną metodę pomocniczą (np. „*createMyStrategy*”), która wyciągnie niezbędne parametry z mapy params i zwróci instancję Twojej klasy.
- **Wyjścia (outputs):** Klasy implementujące interfejs „*OutputStrategy*”. Metoda „*apply*” modyfikuje parametry agenta na podstawie tablicy bool neuronów z informacją które strzeliły. Aktualnie zaimplementowana jest „*PopulationDriveStrategy*” (poruszanie i skręcanie agenta).
  Jak dodać własny typ wyjścia:
  1. Utwórz klasę implementującą „*OutputStrategy*”.
  2. W klasie „*OutputStrategyFactory*” dodaj nowy case w metodzie switch. Powinien on mapować unikalny klucz z pliku konfiguracyjnego (np. "MOJE_WYJSCIE") na dedykowaną metodę tworzącą obiekt Twojej klasy.
  3. Stwórz prywatną metodę pomocniczą (np. „*createMyOutputStrategy*”), która wyciągnie niezbędne parametry z mapy params i zwróci instancję Twojej klasy.

Agent i świat:

Klasa „*Agent*” integruje „*SNNEngine*” z fizyczną reprezentacją agenta. W konstruktorze następuje dynamiczna inicjalizacja systemów wejść i wyjść („*InputSystem*”, „*OutputSystem*”) na podstawie konfiguracji przekazanej w obiekcie „*SnnNetworkData*”.

Cykl życia agenta (metoda „update”) realizowany jest w trzech krokach:

1. Pobranie danych sensorycznych ze świata i konwersja na prądy wejściowe.
2. Wykonanie kroku symulacji sieci neuronowej.
3. Interpretacja aktywności neuronów wyjściowych i aktualizacja pozycji (metoda „move”).

Dodatkowo agent posiada mechanizm nagradzania („applyReward”), który wstrzykuje dopaminę do systemu w momencie zdobycia pożywienia.

Klasa „*World*” zarządza środowiskiem symulacji, przechowując instancję agenta oraz listę obiektów pożywienia („*Food*”). Odpowiada za wczytanie topologii sieci (poprzez „*NetworkTopologyLoader*”) oraz inicjalizację elementów na planszy.

Główna pętla logiczna świata obsługuje:

- Detekcję kolizji agenta z jedzeniem (usunięcie obiektu, przyznanie nagrody, generowanie nowego pożywienia w losowym miejscu).
- Obsługę granic świata („*handleBoundaries*”) – implementacja fizyki odbić, zapobiegająca wyjściu agenta poza obszar symulacji.
- Delegowanie aktualizacji stanu do agenta.

7. Instrukcja wdrożeniowa

Środowisko: JDK 21+, IntelliJ IDEA

Uruchomienie projektu:

1. **Sklonuj repozytorium:**
   ```
   git clone https://github.com/MaciejRozpedek/SNN-Mouse-Simulation-JAVA.git
   ```
2. **Otwórz projekt w IntelliJ IDEA**
3. **Ustaw SDK:** Upewnij się, że w *File* → *Project Structure* → *Project* → *SDK* wybrany jest *JDK 21*.
4. **Uruchom aplikację:** Znajdź klasę „*SnnMouseSimulationApplication*” w folderze „*src/main/java/com.macroz.snnmousesimulation*” i kliknij **Run**.
5. **Otwórz symulację:** Wejdź w przeglądarce na adres: „*http://localhost:8080*”

8. Instrukcja użytkownika

Aplikacja umożliwia symulację i wizualizację autonomicznego agenta sterowanego impulsową siecią neuronową. Konfiguracja sieci odbywa się poprzez plik YAML.

Kroki użycia:

1. **Uruchom symulację:** W przeglądarce kliknij „Start Simulation" aby rozpocząć symulację i „Stop Simulation" aby zatrzymać.
2. **Modyfikacja parametrów:** edytuj plik „*SNNConfig.yaml*". Po zmianach uruchom aplikację ponownie aby załadować nową konfigurację.
3. **Sterowanie i obserwacja:** używaj suwaka SPEED aby regulować tempo symulacji. Obserwuj działanie agenta w środowisku.

9. Podsumowanie i wnioski

- Projekt stanowi solidną bazę do dalszych badań nad impulsowymi sieciami neuronowymi, umożliwiając łatwą rozbudowę o nowe moduły sensoryczne i motoryczne.
- Praca nad implementacją wykazała, że architektura modularna skutecznie oddziela logikę SNN od środowiska symulacyjnego, co ułatwia testowanie.
- Zastosowanie plików YAML do konfiguracji sieci znacząco uprościło proces testowania różnych wariantów architektury bez konieczności ingerencji w kod źródłowy.
- Proces testowania symulacji pokazał, że dobranie parametrów sieci tak, aby agent zachowywał się zgodnie z oczekiwaniami, jest procesem skomplikowanym i wymaga wielu prób.

10. Bibliografia

[1] E. M. Izhikevich, „Simple Model of Spiking Neurons," *IEEE Transactions on Neural Networks,* tom 14, nr 6, pp. 1569-1572, Listopad 2003.